CAPACITACIÓN TÉCNICA · BANCO DE MÉXICO

Desarrollo Seguro de Sistemas

Módulo 2 · A01

Control de Acceso y Configuración Incorrecta de Seguridad



Java JDK 17 | Jakarta EE 11 | PostgreSQL 16

Instructor: Juan Carlos Trejo Herrera · Material de apoyo

APERTURA

Módulo 2. A01 — Control de Acceso

Desarrollo Seguro de Sistemas (OWASP Top 10:2025)

Banco de México · Capacitación técnica

Java JDK 17 · Jakarta EE 11 · PostgreSQL 16

CASO FINANCIERO HIPOTÉTICO

Material de apoyo para el instructor Juan Carlos Trejo Herrera

APERTURA

Autenticación y autorización: no son lo mismo



Autenticación: determina quién presenta la identidad.

Autorización: decide qué puede hacer esa identidad sobre un recurso.

Una sesión autenticada no implica permiso para cualquier operación.

La autorización debe considerar identidad, acción, recurso y contexto.

Pregunta clave

- ¿Quién eres? = autenticación
- ¿Qué puedes hacer aquí? = autorización

CASO FINANCIERO HIPOTÉTICO

Un operador autenticado puede consultar sus operaciones asignadas, pero no aprobar una operación fuera de su ámbito.

A01:2025 — definición y posición

A01 mantiene el primer lugar del OWASP Top 10:2025.

El control de acceso aplica políticas para impedir acciones fuera de los permisos previstos.

Las fallas pueden divulgar, modificar o destruir datos o ejecutar funciones de negocio fuera de los límites del usuario.

La categoría incluye 40 CWE mapeadas en la edición 2025.

Qué protege

- Datos y registros
- Funciones de negocio
- Límites entre usuarios y roles

CASO FINANCIERO HIPOTÉTICO

Una API permite que un usuario consulte o modifique una operación solo cambiando su identificador.

Fallo de mínimo privilegio y denegar por defecto



El acceso se concede solo a capacidades, roles o usuarios que lo necesitan.

Los recursos no públicos deben denegar el acceso salvo autorización explícita.

No asumir que un usuario autenticado tiene permiso por defecto.

Aplicar la misma política a cada método HTTP y operación.

Regla de diseño

- Denegar por defecto
- Conceder lo mínimo
- Revisar cada solicitud

CASO FINANCIERO HIPOTÉTICO

El perfil de consulta no puede invocar PUT /operaciones/{id} ni DELETE /operaciones/{id}.

Eludir controles mediante manipulación de URL



Cambiar rutas, parámetros o identificadores puede revelar controles ausentes.

Ocultar un enlace en la interfaz no protege el endpoint.

La navegación forzada consiste en solicitar directamente rutas no enlazadas o restringidas.

La comprobación debe ejecutarse en el servidor.

Prueba controlada

- Modificar el identificador
- Repetir la petición
- Comparar autorización y respuesta

CASO FINANCIERO HIPOTÉTICO

Un usuario cambia /operaciones/1001 por /operaciones/1002 e intenta consultar una operación de otro participante.

IDOR: referencias directas inseguras a objetos

IDOR aparece cuando un identificador controlado por el usuario permite acceder a un objeto sin validar autorización.

Los identificadores secuenciales facilitan la enumeración, pero usar UUID no reemplaza la autorización.

Validar propiedad o permiso sobre el registro en cada operación.

La regla debe aplicarse también a lectura, actualización y eliminación.

Evidencia

- Identidad autenticada
- ID solicitado
- Propietario del recurso
- Decisión de autorización

CASO FINANCIERO HIPOTÉTICO

La API devuelve el detalle de una operación de otro usuario al modificar el parámetro operationId.

Autorización por método HTTP



No basta con proteger GET; POST, PUT, PATCH y DELETE requieren controles equivalentes.

Las rutas y métodos deben mapearse a acciones de negocio autorizadas.

No confiar en que la interfaz deshabilita botones o formularios.

Probar combinaciones de rol, recurso y método.

Matriz de pruebas

- GET · consulta
- POST · creación
- PUT/PATCH · cambio
- DELETE · eliminación

CASO FINANCIERO HIPOTÉTICO

Un operador puede consultar una operación mediante GET, pero el servidor también acepta DELETE sin verificar privilegios.

Elevación horizontal y vertical de privilegios



Horizontal: acceder a recursos de otro usuario o entidad con privilegios similares.

Vertical: ejecutar funciones reservadas a un rol superior.

La elevación puede ocurrir con usuario autenticado o sin sesión.

Separar autorización de función y autorización de objeto.

Dos preguntas

- ¿Puede ejecutar esta acción?
- ¿Puede ejecutarla sobre este objeto?

CASO FINANCIERO HIPOTÉTICO

Un operador consulta operaciones de otro participante (horizontal) o accede a la función de aprobación de supervisor (vertical).

CORS mal configurado como riesgo de acceso



Una política CORS permisiva puede permitir acceso desde orígenes no confiables.

CORS es una política aplicada por navegadores; no sustituye autorización del servidor.

No reflejar indiscriminadamente el encabezado Origin.

Limitar orígenes, métodos y encabezados a lo requerido.

Validación

- Origen permitido explícito
- Métodos mínimos
- Credenciales bajo política estricta

CASO FINANCIERO HIPOTÉTICO

Una API financiera responde con Access-Control-Allow-Origin: * y permite credenciales de forma incorrecta o refleja cualquier origen.

SSRF dentro de A01:2025

OWASP 2025 incorpora SSRF dentro de A01.

La aplicación puede ser inducida a realizar solicitudes a destinos no previstos.

El servidor puede alcanzar recursos internos que el usuario externo no podría consultar directamente.

Validar y restringir destinos, protocolos y redirecciones según el caso.

Riesgo

- El servidor actúa como intermediario
- La red interna puede quedar expuesta

CASO FINANCIERO HIPOTÉTICO

Un endpoint de consulta de comprobantes acepta una URL y el servidor intenta recuperar recursos de una dirección interna.

Centralizar la autorización

Evitar comprobaciones improvisadas y distintas en cada endpoint.

Definir políticas reutilizables y declarativas.

Mantener consistencia entre rutas y métodos HTTP.

Registrar fallos de autorización y alertar cuando el patrón sea relevante.

Ventaja

- Menos duplicación
- Menos omisiones
- Comportamiento consistente

CASO FINANCIERO HIPOTÉTICO

Un componente común de autorización verifica permisos de consulta, aprobación y cancelación en todos los recursos.

Denegar por defecto y proteger rutas

Definir explícitamente los recursos públicos.

Exigir identidad y permiso para recursos privados.

Revisar rutas administrativas y endpoints no enlazados en la interfaz.

Evitar que una nueva ruta quede expuesta por ausencia de una regla.

Prueba negativa

- Sin sesión → denegado
- Rol incorrecto → denegado
- Rol correcto → permitido

CASO FINANCIERO HIPOTÉTICO

Una ruta /admin/reportes no debe estar disponible para el rol consulta aunque el usuario conozca o adivine la URL.

Sesión: vínculo entre solicitudes



Una sesión permite mantener contexto de identidad entre solicitudes HTTP.

El identificador de sesión actúa como credencial temporal.

Si se roba o abusa de la sesión, el atacante puede actuar con los privilegios asociados.

Proteger creación, transporte, uso y finalización de la sesión.

Ciclo

- Crear
- Usar
- Renovar según política
- Invalidar

CASO FINANCIERO HIPOTÉTICO

Una cookie de sesión expuesta puede permitir que un tercero opere con la identidad de un usuario.

Invalidación de sesión y tokens

OWASP indica invalidar en servidor los identificadores de sesión con estado al cerrar sesión.

Los JWT sin estado deben ser de corta duración para reducir la ventana de oportunidad.

Para JWT de mayor duración, considerar refresh tokens y estándares OAuth para revocación.

El cierre de sesión debe tener un efecto real en el servidor.

FRAGMENTO ILUSTRATIVO

```
// Pseudocódigo de cierre de sesión
sesiones.invalidar(idSesion);
respuesta.cookie("SESSIONID", "")
      .httpOnly(true)
      .secure(true)
      .maxAge(0);
```

CASO FINANCIERO HIPOTÉTICO

Tras cerrar sesión en el portal de consulta, reutilizar el identificador anterior no debe permitir acceder a operaciones.

Atributos de cookies de sesión

Secure: limita el envío de la cookie a conexiones HTTPS.

HttpOnly: impide que JavaScript lea la cookie directamente.

SameSite: controla el envío de cookies en contextos entre sitios.

Los atributos deben definirse de acuerdo con el flujo y la política de sesión.



Revisión

- Secure
- HttpOnly
- SameSite

CASO FINANCIERO HIPOTÉTICO

La cookie de sesión del portal financiero debe tener atributos coherentes con el uso HTTPS y el modelo de navegación.

¿Qué es CSRF?

Cross-Site Request Forgery: un sitio externo induce al navegador autenticado a enviar una solicitud a otro sitio.

El navegador puede adjuntar automáticamente cookies de sesión al destino.

El servidor podría interpretar la solicitud como acción legítima del usuario.

A01:2025 incluye CWE-352 dentro de Pérdida de Control de Acceso.

Condición

- Navegador autenticado
- Solicitud inducida
- Servidor confía solo en cookie

CASO FINANCIERO HIPOTÉTICO

Un usuario con sesión abierta visita una página externa que intenta provocar una transferencia o cambio de datos en otro servicio.

Flujo conceptual de un ataque CSRF

El usuario inicia sesión en la aplicación legítima.

La cookie de sesión queda almacenada en el navegador.

El usuario visita un sitio externo controlado por un atacante.

El sitio induce una petición al servicio legítimo y el navegador puede adjuntar la cookie.



Flujo

- 1 · Sesión activa
- 2 · Sitio externo
- 3 · Petición inducida
- 4 · Cookie adjunta

CASO FINANCIERO HIPOTÉTICO

Escenario ilustrativo: una solicitud de cambio de beneficiario se envía sin que el usuario haya iniciado esa acción conscientemente.

Defensas contra CSRF



Usar tokens anti-CSRF impredecibles y vinculados a la sesión o contexto.

Validar el token en solicitudes que cambian estado.

Configurar SameSite como defensa adicional, no como única defensa universal.

Verificar Origin/Referer cuando aplique y diseñar operaciones sensibles para evitar cambios por GET.

Defensa en capas

- Token CSRF
- SameSite
- Validación de origen
- Métodos adecuados

CASO FINANCIERO HIPOTÉTICO

El endpoint de actualización de datos de participante exige un token CSRF válido además de la sesión autenticada.

CSRF: qué debe probarse

Solicitudes legítimas con token válido deben funcionar.

Solicitudes sin token, con token incorrecto o de otra sesión deben rechazarse.

Los endpoints que cambian estado no deben aceptar GET.

Probar flujos reales de formularios y APIs con credenciales/cookies.

Resultado esperado

- Solicitud legítima: permitida
- Solicitud forjada: rechazada

CASO FINANCIERO HIPOTÉTICO

El equipo intenta enviar una solicitud de modificación desde un origen no confiable y verifica que el servidor la rechace.